📚 Inheritance Basics

> **Inheritance** is a core OOP feature that enables hierarchical classification. A *base class* defines common traits, while *derived classes* inherit those traits and add specific features.

- **Base class** – the class being inherited.
- **Derived class** – the class that inherits; it can itself become a base for further inheritance (multilevel).
- When a class inherits, the base's members become members of the derived class.

General syntax

```
class Derived : access_specifier Base {
    // body of derived class
};
```

- access_specifier can be **public**, **protected**, or **private**.
- Default: private for class, public for struct.

🔗 Public Inheritance

> With **public inheritance**, the access levels of base members are preserved in the derived class.

Base member type	Derived access level
public	public
protected	protected
private	*inaccessible*

Example

```
#include <iostream>
using namespace std;

class Base {
    int i, j;                       // private
public:
    void set(int a, int b) { i = a; j = b; }
    void show() { cout << i << " " << j << "\n"; }
};
```

```
class Derived : public Base {
    int k;
public:
    Derived(int x) { k = x; }
    void showk() { cout << k << "\n"; }
};

int main() {
    Derived ob(3);
    ob.set(1, 2);
    ob.show();   // from Base
    ob.showk();  // from Derived
    return 0;
}
```

Output

```
1 2
3
```

🔐 Protected Members

> A **protected** member is hidden from non-member functions and objects but visible to the class itself and any derived class (when inheritance is public or protected).

- Acts like private data for the rest of the program while remaining inheritable.

Example

```
#include <iostream>
using namespace std;

class Base {
protected:
    int i, j;
public:
    void set(int a, int b) { i = a; j = b; }
    void show() { cout << i << " " << j << "\n"; }
};

class Derived : public Base {
    int k;
public:
```

```
    void setk() { k = i * j; }   // direct access to i, j
    void showk() { cout << k << "\n"; }
};

int main() {
    Derived ob;
    ob.set(2, 3);
    ob.show();   // prints 2 3
    ob.setk();
    ob.showk();  // prints 6
    return 0;
}
```

Output

```
2 3
6
```

📚 Multilevel Inheritance & Protected

> In **multilevel inheritance**, protected members remain protected throughout the chain, allowing each derived level to use them directly.

Example

```
#include <iostream>
using namespace std;

class Base {
protected:
    int i, j;
public:
    void set(int a, int b) { i = a; j = b; }
    void show() { cout << i << " " << j << "\n"; }
};

class Derived1 : public Base {
    int k;
public:
    void setk() { k = i * j; }
    void showk() { cout << k << "\n"; }
};
```

```
class Derived2 : public Derived1 {
    int m;
public:
    void setm() { m = i - j; }
    void showm() { cout << m << "\n"; }
};

int main() {
    Derived1 d1;
    Derived2 d2;

    d1.set(2, 3);
    d1.show();   // 2 3
    d1.setk();
    d1.showk();  // 6

    d2.set(3, 4);
    d2.show();   // 3 4
    d2.setk();
    d2.setm();
    d2.showk();  // 12
    d2.showm();  // -1
    return 0;
}
```

Output

```
2 3
6
3 4
12
-1
```

🌐 Multiple Inheritance

> A derived class can inherit from **multiple base classes** by listing them comma-separated, each with its own access specifier.

Example

```
#include <iostream>
using namespace std;

class Base1 {
protected:
    int x;
public:
    void showx() { cout << x << "\n"; }
};

class Base2 {
protected:
    int y;
public:
    void showy() { cout << y << "\n"; }
};

class Derived : public Base1, public Base2 {
public:
    void set(int i, int j) { x = i; y = j; }
};

int main() {
    Derived ob;
    ob.set(10, 20);
    ob.showx();   // from Base1
    ob.showy();   // from Base2
    return 0;
}
```

Output

```
10
20
```

🏗️ Constructor & Destructor Order

> **Construction** proceeds from the most base class upward; **destruction** proceeds in reverse order.

Example (multilevel)

```
#include <iostream>
using namespace std;

class Base {
public:
    Base()  { cout << "Constructing base\n"; }
    ~Base() { cout << "Destructing base\n"; }
};

class Derived1 : public Base {
public:
    Derived1()  { cout << "Constructing derived1\n"; }
    ~Derived1() { cout << "Destructing derived1\n"; }
};

class Derived2 : public Derived1 {
public:
    Derived2()  { cout << "Constructing derived2\n"; }
    ~Derived2() { cout << "Destructing derived2\n"; }
};

int main() {
    Derived2 obj;
    return 0;
}
```

Output

```
Constructing base
Constructing derived1
Constructing derived2
Destructing derived2
Destructing derived1
Destructing base
```

📩 Passing Arguments to Base Constructors

> When a base class has a **parameterized constructor**, a derived class must forward arguments using an **initialization list**.

Syntax

```
Derived(args) : Base(base_args) { /* body */ }
```

Example

```
#include <iostream>
using namespace std;

class Base {
protected:
    int i;
public:
    Base(int x) { i = x; cout << "Constructing base\n"; }
    ~Base() { cout << "Destructing base\n"; }
};

class Derived : public Base {
    int j;
public:
    Derived(int x, int y) : Base(y) { j = x; cout << "Constructing derived\n"; }
    ~Derived() { cout << "Destructing derived\n"; }
    void show() { cout << i << " " << j << "\n"; }
};

int main() {
    Derived ob(3, 4);
    ob.show();   // prints 4 3
    return 0;
}
```

Output

```
Constructing base
Constructing derived
4 3
Destructing derived
Destructing base
```

🔓 Restoring Access after Private Inheritance

> Private inheritance turns all inherited public and protected members into **private**. Select members can be re-exposed using a **using-declaration** (preferred) or the older **access**

> **declaration**.

Rules

- Only public or protected members may be restored.
- Private members of the base cannot be made public/protected.

Conceptual Example (using-declaration)

```
class Base {
public:
    int j;
};

class Derived : private Base {
public:
    using Base::j;   // j becomes public in Derived
};
```

(The same effect can be achieved with Base::j; placed under a public: section, but that form is deprecated.)

🌀 Virtual Members & Polymorphism

> **Virtual members** enable **run-time polymorphism**: a base-class pointer or reference can invoke the most-derived implementation of a function.

- Requires the function to be declared virtual in the base class.
- The actual function is selected via the **virtual table (v-table)** at execution time.

🏛️ Virtual Base Classes

> In multiple inheritance, **virtual base classes** ensure that only **one** shared subobject of the base exists, eliminating ambiguity.

Example

```
#include <iostream>
using namespace std;
```

```
class Base {
public:
    int i;
};

class Derived1 : virtual public Base { public: int j; };
class Derived2 : virtual public Base { public: int k; };

class Derived3 : public Derived1, public Derived2 {
public:
    int sum;
};

int main() {
    Derived3 obj;
    obj.i = 10;   // single shared Base subobject
    obj.j = 20;
    obj.k = 30;
    obj.sum = obj.i + obj.j + obj.k;
    cout << obj.i << " " << obj.j << " " << obj.k << " " << obj.sum;
    return 0;
}
```

Output

```
10 20 30 60
```

📞 Virtual Functions

> A **virtual function** is a non-static member declared with virtual in a base class and optionally overridden in derived classes.

- Must have identical signature in all overrides.
- Destructors can (and often should) be virtual.
- Constructors cannot be virtual.

Example

```
#include <iostream>
using namespace std;

class Base {
public:
```

```
    virtual void vfunc() { cout << "This is base's vfunc().\n"; }
};

class Derived1 : public Base {
public:
    void vfunc() override { cout << "This is derived1's vfunc().\n"; }
};

class Derived2 : public Base {
public:
    void vfunc() override { cout << "This is derived2's vfunc().\n"; }
};

int main() {
    Base *p;
    Base b;
    Derived1 d1;
    Derived2 d2;

    p = &b;  p->vfunc();
    p = &d1; p->vfunc();
    p = &d2; p->vfunc();
    return 0;
}
```

Output

```
This is base's vfunc().
This is derived1's vfunc().
This is derived2's vfunc().
```

📣 Calling Virtual Functions via References

> A **base-class reference** can bind to a base or any derived object. When the referenced type has a virtual function, the call resolves to the most-derived override.

Example

```
#include <iostream>
using namespace std;

class Base {
```

```
public:
    virtual void vfunc() { cout << "This is base's vfunc().\n"; }
};

class Derived1 : public Base {
public:
    void vfunc() override { cout << "This is derived1's vfunc().\n"; }
};

class Derived2 : public Base {
public:
    void vfunc() override { cout << "This is derived2's vfunc().\n"; }
};

void f(Base &r) { r.vfunc(); }

int main() {
    Base b;
    Derived1 d1;
    Derived2 d2;
    f(b);
    f(d1);
    f(d2);
    return 0;
}
```

Output

```
This is base's vfunc().
This is derived1's vfunc().
This is derived2's vfunc().
```

📜 Rules for Virtual Functions

> - Must be a **non-static** member of a class.
> - Cannot be a constructor; destructors may be virtual.
> - Must have a definition in the base class (even if never called).
> - All overrides must match the base signature exactly.
> - A base pointer can point to any derived object; the reverse is not allowed.
> - If a derived class does **not** override, the base implementation is used.

🗝️ Pure Virtual Functions & Abstract Classes

> A **pure virtual function** (= 0) provides no implementation in the base class, making the class **abstract**.

- Abstract classes cannot be instantiated.
- Derived classes must override every pure virtual function to become concrete.

Example

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() = 0;   // pure virtual
};

class Derived : public Base {
public:
    void show() override { cout << "Derived class show().\n"; }
};

int main() {
    Derived d;
    d.show();
    return 0;
}
```

Output

```
Derived class show().
```

🏛️ Abstract Class Overview

> **Abstract class**: a class containing at least one pure virtual function.

- Cannot create objects of the abstract type.
- Can declare pointers/references to the abstract type for polymorphic use.

Rules

Rule	Description
No objects	Direct instantiation is prohibited.
Can hold pointers/references	Enables run-time polymorphism.
Must override	Every pure virtual function must be overridden in concrete derived classes.

⚖️ Polymorphism Types

> **Polymorphism** – same interface, different implementations.

Type	Mechanism	Binding time
Static (Compile-time)	Function/operator overloading	Compile time
Dynamic (Run-time)	Virtual functions, abstract base classes	Run time

---

📊 Static vs Dynamic Polymorphism Table

Feature	Static Polymorphism	Dynamic Polymorphism
Binding	Compile-time (early)	Run-time (late)
Implementation	Overloading, templates	Virtual functions, inheritance
Performance	Faster (no v-table lookup)	Slight overhead (v-table)
Flexibility	Limited – fixed at compile	Highly flexible – can add new derived types
Typical Use	Simple utility functions	Frameworks, plug-in architectures

---

📌 Pointers to Objects

> Objects can be accessed via pointers using the **arrow (->)** or **dereference (*)** operators.

Example

```
class Alpha {
    int x;
public:
    void set(int i) { x = i; }
    void show() { cout << x << "\n"; }
};

int main() {
    Alpha obj;
    Alpha *ptr = &obj;
    ptr->set(10);
    ptr->show();               // arrow syntax
    (*ptr).show();             // dereference + dot
    return 0;
}
```

Output

```
10
```

🧭 The this Pointer

> Inside a non-static member function, this is a pointer to the object on which the function was invoked.

- Allows disambiguation of member names and enables method chaining (return *this;).

Example

```
class Person {
    int age;
public:
    Person(int a) { age = a; }
    Person& older(Person &other) {
        if (other.age > age) return other;
        return *this;
    }
};
```

📎 Pointers to Derived Classes

> A base-class pointer can hold the address of any derived object, enabling polymorphic behavior.

Example

```
class Polygon {
protected:
    int w, h;
public:
    void set(int a, int b) { w = a; h = b; }
};

class Rectangle : public Polygon {
public:
    int area() { return w * h; }
};

class Triangle : public Polygon {
public:
    int area() { return w * h / 2; }
};

int main() {
    Rectangle rect;
    Triangle tri;
    Polygon *p1 = &rect;
    Polygon *p2 = &tri;

    p1->set(4,5);
    p2->set(4,5);
    cout << rect.area() << "\n";
    cout << tri.area() << "\n";
    return 0;
}
```

Output

```
20
10
```

🎛️ Default Function Arguments

> Default arguments provide fallback values when callers omit parameters.

- Must appear **trailing** in the parameter list.
- Specified in the function **declaration**, not the definition.

Example

```
#include <iostream>
using namespace std;

void showMessage(string msg = "Hello", int times = 1) {
    for (int i = 0; i < times; ++i)
        cout << msg << endl;
}

int main() {
    showMessage();               // uses both defaults
    showMessage("Welcome");      // default for times
    showMessage("Bye", 3);       // no defaults
    return 0;
}
```

Output

```
Hello
Welcome
Bye
Bye
Bye
```

➕ Operator Overloading

> Overloading lets user-defined types use built-in operators, improving readability.

Example – Complex Number Addition

```
#include <iostream>
using namespace std;

class Complex {
    float real, imag;
```

```
public:
    Complex(float r = 0, float i = 0) : real(r), imag(i) {}
    Complex operator+(Complex o) {
        return Complex(real + o.real, imag + o.imag);
    }
    void display() { cout << real << " + " << imag << "i\n"; }
};

int main() {
    Complex c1(2.5, 3.5), c2(1.6, 2.4);
    Complex result = c1 + c2;
    cout << "Sum = ";
    result.display();
    return 0;
}
```

Output

```
Sum = 4.1 + 5.9i
```

➕➖ Overloading ++ and -- Operators (Postfix)

> Postfix increment/decrement can be overloaded via a dummy int parameter.

Example

```
#include <iostream>
using namespace std;

class Number {
    int x;
public:
    Number(int a = 0) : x(a) {}
    friend Number operator++(Number &obj, int);
    friend Number operator--(Number &obj, int);
    void show() const { cout << "x = " << x << endl; }
};

Number operator++(Number &obj, int) {
    Number temp = obj;
    obj.x += 1;
    return temp;
}
```

```
Number operator--(Number &obj, int) {
    Number temp = obj;
    obj.x -= 1;
    return temp;
}

int main() {
    Number n(5);
    cout << "Start: "; n.show();
    Number a = n++;   // n becomes 6
    cout << "After n++: "; n.show();
    cout << "Returned from n++: "; a.show();
    Number b = n--;   // n becomes 5
    cout << "After n--: "; n.show();
    cout << "Returned from n--: "; b.show();
    return 0;
}
```

Output

```
Start: x = 5
After n++: x = 6
Returned from n++: x = 5
After n--: x = 5
Returned from n--: x = 6
```